SYSTEMS PAPER · PREPRINT

SAFIR: Edge-basierte medizinische Multi-Patienten-Diktierpipeline auf Basis kleiner Sprachmodelle

Ein Systems-Paper zum Defense-in-Depth-Post-Processing für feldtaugliche Spracherkennung

SAFIR-Entwicklungsteam

CGI Deutschland GmbH · Bundeswehr Sanitätsdienst

Version 2.1-preprint · April 2026

ZUSAMMENFASSUNG (ABSTRACT)

Wir stellen SAFIR vor, eine edge-basierte Spracherkennungs- und Extraktionspipeline für die militärische medizinische Felddokumentation. Das System kombiniert OpenAI Whisper (large-v3-turbo) zur automatischen Spracherkennung (ASR) mit Google Gemma 3 4B für die nachgelagerte Multi-Patienten-Segmentierung und Feldextraktion. Beide Modelle laufen vollständig auf einem 7.4 GB NVIDIA Jetson Orin Nano unter einem Leistungsbudget von 15 W. Eine neuartige vierstufige, deterministische Post-Merge-Strategie härtet die nicht-deterministische LLM-Ausgabe gegen typische Fehlermuster ab (pronominale Abschnittsgrenzen, Einleitungsphrasen, inhaltsleere Fragmente und Meta-Kommunikation mitten im Diktat). Wir evaluieren die Robustheit des Systems an adversariellen Testtranskripten, die gezielt Small-Talk-Einschübe, unterbrochene Patienten-Einleitungen und stark akzentuierte Diktate enthalten. Die Ergebnisse zeigen: Die Pipeline produziert in allen Testfällen **null halluzinierte Patienten** — sie tauscht Recall gegen Precision in Übereinstimmung mit der medizinischen Anforderung „Niemals einen Patienten erfinden". Wir diskutieren die Design-Philosophie, die architektonischen Kompromisse, die ethischen Implikationen des Einsatzes von LLMs in Hochrisiko-Entscheidungsunterstützungskontexten sowie Richtungen für zukünftige Arbeit — darunter Prompt-Engineering für konversationelle Diktate und Integration retrieval-gestützter Konfidenzbewertung.

Schlagworte: Edge-KI, automatische Spracherkennung, große Sprachmodelle, medizinisches NLP, militärische Gesundheitsinformatik, Defense-in-Depth, konservative Halluzinationsvermeidung, On-Device-Inferenz, Prompt-Engineering, Rettungskette der Bundeswehr

INHALT

1. Einführung
2. Verwandte Arbeit
3. Systemarchitektur
4. Methodologie
5. Designentscheidungen und Kompromisse
6. Empirische Evaluation
7. Diskussion
8. Zukünftige Arbeit
9. Schlussfolgerung
10. Danksagung

Anhang A: Vollständiger BOUNDARY_PROMPT · **Anhang B:** Referenzen · **Anhang C:** Code-Verfügbarkeit

1. Einführung

1.1 Problemstellung

Die militärische Feldmedizin operiert unter dem von der NATO-Doktrin definierten Begriff der **„Goldenen Stunde"** — dem 60-Minuten-Fenster zwischen Verwundung und definitiver medizinischer Versorgung, in dem die Überlebenschance des Patienten maximiert wird. Innerhalb dieses Fensters triagiert ein einzelner Sanitäter (Combat Medic) möglicherweise mehrere Verwundete, initiiert Behandlungen und muss eine Dokumentation erstellen, die den Patienten durch vier aufeinanderfolgende Versorgungsstufen begleitet (Phase 0 Selbsthilfe bis Role 4 Rehabilitation).

Die traditionelle Dokumentation erfolgt über papierbasierte TCCC-Karten (Tactical Combat Casualty Care). Dieser Ansatz hat mehrere dokumentierte Probleme:

1. **Kognitive Last:** Der Sanitäter schreibt, während er gleichzeitig behandelt
2. **Lesbarkeit:** Feldbedingungen (Regen, Schmutz, Blut) beeinträchtigen die Handschrift
3. **Transkriptionsfehler:** Papierdaten müssen an der Rettungsstation manuell übertragen werden
4. **Kein Echtzeit-Lagebild:** Kommandostellen erfahren erst Minuten oder Stunden später von Verwundeten

Eine sprachgestützte Alternative löst alle vier Probleme, bringt aber neue Herausforderungen mit sich:

1. **Konnektivität:** Gefechtseinsätze haben oft kein stabiles Netz für Cloud-basierte ASR/NLP
2. **Multi-Patient-Diktat:** Ein einzelnes Sanitäter-Sprachmemo kann mehrere Verwundete beschreiben
3. **Domänenspezifisches Vokabular:** Militärische Dienstgrade, medizinische Fachbegriffe, Medikamentennamen
4. **Hohe Einsätze:** Ein halluzinierter Patient oder erfundene Vitalwerte können Schaden verursachen
5. **Sprache:** Bundeswehr-Sanitäter arbeiten auf Deutsch, einer ressourcenärmeren Sprache für medizinisches NLP

SAFIR ist unsere Antwort: eine vollständige, offline-fähige Pipeline auf einem 15-W-Edge-Gerät, die explizit auf dem Prinzip aufbaut, dass **das Auslassen von Informationen stets dem Erfinden vorzuziehen ist**.

1.2 Beiträge dieses Papers

Dieses Paper leistet die folgenden Beiträge:

1. Eine produktiv eingesetzte Systemarchitektur für edge-residente medizinische Sprach-zu-Datensatz-Verarbeitung auf ressourcenbeschränkter Hardware (7.4 GB unified memory). Insbesondere lösen wir das Koexistenzproblem von Whisper und LLM durch explizite Swap-Mode-Orchestrierung
2. Ein vierstufiger deterministischer Post-Merge-Algorithmus, der LLM-generierte Patientengrenzen-Vorhersagen gegen drei unterschiedliche Fehlermuster härtet (Pronomen-Fortsetzung, Einleitungsphrasen, inhaltsleere Fragmente)
3. Eine Prompt-Engineering-Methodologie für deutschsprachige Multi-Patienten-Diktate, einschließlich eines getesteten Templates mit vier kanonischen Few-Shot-Beispielen
4. Eine empirische Robustheits-Evaluation gegen adversarielle Transkripte, die gezielt Intro-Filterung, Irrelevanz-Filterung und unterbrochene Rede testen
5. Eine explizite Formulierung der Design-Philosophie „Konservative Halluzinationsvermeidung" für medizinische ML-Domänen

1.3 Struktur des Papers

Abschnitt 2 gibt einen Überblick über verwandte Arbeit in medizinischem NLP, On-Device-Inferenz und LLM-Post-Processing. Abschnitt 3 beschreibt die Systemarchitektur. Abschnitt 4 beschreibt detailliert die Methodologie. Abschnitt 5 diskutiert wichtige Designentscheidungen und deren Kompromisse. Abschnitt 6 präsentiert die Robustheits-Evaluation. Abschnitt 7 diskutiert Limitierungen, ethische Überlegungen und offene Fragen. Abschnitt 8 skizziert zukünftige Arbeit. Abschnitt 9 schließt.

2. Verwandte Arbeit

2.1 Medizinische Spracherkennung

Kommerzielle Diktiersysteme wie Nuance Dragon Medical One (Microsoft) dominieren die klinische Transkription, sind jedoch cloud-basiert, benötigen stabile Konnektivität und bieten nur begrenzte Anpassung für nicht-englische militärische Kontexte [1, 2]. Domänen-angepasste Whisper-Varianten wurden für die englische klinische Diktate publiziert [3], jedoch bleiben deutsche medizinische Korpora für Fine-Tuning aufgrund von Datenschutzbeschränkungen knapp.

Unser Ansatz unterscheidet sich in drei Punkten: (1) Wir nutzen Whisper unverändert (large-v3-turbo) und verlassen uns für die Domänenadaption auf LLM-Post-Processing, (2) wir arbeiten vollständig offline und (3) wir verarbeiten Multi-Patienten-Einzel-Sessions — ein Szenario, das in der Massenanfall-Triage häufig vorkommt, aber in bestehenden klinischen Systemen nicht abgedeckt wird.

2.2 LLM-basierte Informationsextraktion

Aktuelle Arbeit zeigt, dass kleinere LLMs (unter 10 Mrd. Parameter) bei strukturierter Extraktion konkurrenzfähige Leistung erreichen können, wenn sie mit sorgfältigem Prompt-Engineering und Post-

Processing kombiniert werden [4, 5]. Prompt-Engineering für medizinisches NLP profitiert erwiesenermaßen von Few-Shot-Beispielen, expliziten Negativbeispielen und Output-Format-Beschränkungen [6].

Unser Beitrag liegt nicht im Modell-Fine-Tuning, sondern in der **Kompositionsschicht**: wie kombiniert man ein kleines LLM (Gemma 3 4B) mit deterministischer Vor- und Nachbearbeitung, um trotz der inhärenten Nicht-Determinismus generativer Modelle Produktions-Zuverlässigkeit zu erreichen.

2.3 Edge-KI für Verteidigungsanwendungen

DARPA-finanzierte Projekte wie die „Squad X"-Initiative haben Voice-to-Text für abgesetzte Operationen erforscht, jedoch fokussieren publizierte Evaluationen auf Keyword-Spotting statt auf freie medizinische Diktate [7]. Das NATO Allied Command Transformation hat edge-basierte KI als prioritären Fähigkeitsbereich identifiziert [8], obwohl konkrete medizinische Anwendungen in der offenen Literatur selten sind.

SAFIR ist unseres Wissens nach die erste öffentlich beschriebene edge-basierte medizinische Dokumentationspipeline, die explizit für eine nationale Streitmacht (Bundeswehr) entwickelt wurde, und deren architektonische Kompromisse transparent für Reproduktion dokumentiert sind.

2.4 Halluzinationsvermeidung in generativen Modellen

Ansätze zur Halluzinationsvermeidung in LLMs lassen sich in drei Kategorien einteilen [9]:

1. **Training-basiert:** RLHF, DPO, Constitutional AI
2. **Inferenz-basiert:** Chain-of-Thought, Self-Consistency, Verifier-Modelle
3. **Post-Processing:** Regelbasierte Filterung, retrieval-gestützte Konsistenzprüfungen

SAFIR wählt den Post-Processing-Ansatz aus praktischen Gründen: Gemma 3 können wir ohne proprietäre Gewichte nicht nachtrainieren, und Inferenz-Techniken (Chain-of-Thought, Self-Consistency) würden die Latenz in einem 15-W-Leistungsbudget vervielfachen. Unser vierstufiges Post-Merge ist eine Instanz regelbasierter Konsistenzerzwingung auf nicht-deterministischer Grenzen-Vorhersage.

3. Systemarchitektur

3.1 Überblick

SAFIR ist ein Zwei-Geräte-System:

1. **Feldgerät (BAT — „Beweglicher Arzt-Trupp"):** NVIDIA Jetson Orin Nano Super, 7.4 GB unified memory, headless Ubuntu mit systemd-Service-Auto-Start. Vom Sanitäter getragen. Bedienung über Mikrofon, zwei GPIO-Taster, ein SSD1306-OLED-Display, einen RC522-RFID-Reader sowie drei Status-LEDs.
2. **Rettungsstation (Role 1):** Microsoft Surface mit Windows + Tailscale. Bedienung über browserbasierte taktische Lagekarte (Leaflet) sowie einen HID-Omnikey-Desktop-RFID-Reader.

Die beiden Geräte kommunizieren ausschließlich über das Tailscale-Mesh-VPN (ein WireGuard-Overlay mit Curve25519-ECDH, ChaCha20-Poly1305 und Blake2s-Hashing — als moderne symmetrische Kryptographie

klassifiziert [10]).

3.2 Software-Stack auf dem Feldgerät

```
┌─ UI/Presentation ──────────────────────────────────────────┐
│ Jinja2-Templates, Vanilla JS, WebSocket für Live-State-Sync  │
├─ Application Layer ────────────────────────────────────────┤
│ FastAPI (uvicorn), app.py mit Domain-Logik für Segmenter,    │
│ Extractor, Patient-Lifecycle, RFID-Batch-Write, Backend-Sync │
├─ Inference Layer ──────────────────────────────────────────┤
│ Whisper (large-v3-turbo via whisper.cpp, GPU-resident)       │
│ Gemma 3 4B (Q4_K_M, via Ollama, GPU-resident wenn aktiv)     │
│ Vosk (dt. Klein-Modell, CPU, ~15 ms Command-Latenz)          │
│ Piper TTS (neural CPU-basiert, Thorsten-high-Stimme)         │
├─ Hardware Abstraction ─────────────────────────────────────┤
│ RC522-SPI-Treiber (shared/rfid.py)                           │
│ SSD1306-I²C-OLED (jetson/oled.py)                            │
│ GPIO-Button-Polling, LED-Pattern-State-Machine               │
└─ OS ───────────────────────────────────────────────────────┘
    NVIDIA JetPack 5.x (Ubuntu 20.04) mit CUDA 12.6
```

3.3 Das Koexistenz-Problem

Eine zentrale architektonische Herausforderung: Whisper (large-v3-turbo quantisiert ~1.2 GB) und Gemma 3 4B (Q4_K_M quantisiert ~4.3 GB) können nicht gleichzeitig im 7.4 GB unified memory resident sein und dabei ~1 GB für CUDA/Tegra-Overhead freilassen.

Wir lösen das durch eine explizite **Swap-Mode-Orchestrierung** auf Applikationsebene:

- **Recording-Phase:** Whisper resident, Gemma entladen. Maximal ~1.2 GB belegt, ~6.2 GB frei
- **Analyse-Phase:** Whisper explizit entladen (`ollama stop`), Gemma geladen mit `keep_alive=-1`. Maximal ~4.3 GB belegt, ~3.1 GB frei
- **Übergänge:** Getriggert durch Anwendungszustandsänderungen (Recording stop → Analyse start). Dauert ~4–8 s für den Modell-Swap

Damit vermeiden wir teurere Hardware, erhalten aber die Einzel-Phase-Latenzcharakteristiken für beide Anwendungsfälle.

4. Methodologie

4.1 Automatische Spracherkennungs-Pipeline

Audio wird bei 16 kHz mono über ein USB-Freisprech-Mikrofon erfasst. Whisper large-v3-turbo läuft als GPU-residenter `whisper.cpp`-Server und verarbeitet das Live-Audio in 25-Sekunden-Chunks mit 2 Sekunden Overlap, um Wortabschneidungen an Chunk-Grenzen zu vermeiden. Die deutsche Sprache wird explizit in den Whisper-Prompt gesetzt:

```
<|startoftranscript|><|de|><|transcribe|>
```

Nach dem ASR wird das resultierende Transkript mithilfe eines deterministischen, regex-basierten Splitters an deutschen Satzend-Zeichen (`.`, `!`, `?`) in Sätze aufgeteilt. Kurze Fragmente (< 30 Zeichen) werden in das vorherige Segment eingefügt, um Mini-Segmente zu vermeiden, die die nachgelagerte LLM-Grenzen-Vorhersage verwirren.

4.2 LLM-basierte Segmentierung

Problemformulierung: Gegeben seien N nummerierte Sätze; sage die Indizes vorher, an denen die Beschreibung eines neuen Patienten beginnt. Beispielsweise für 12 Sätze, die drei Patienten beschreiben, ist die erwartete Ausgabe `{"starts":[0,4,8]}`.

Modell: Gemma 3 4B mit Q4_K_M-Quantisierung (4.3 GB), ausgeliefert von Ollama mit Parametern:

```
options = {
    "num_gpu": -1,        # alle Layer auf GPU
    "temperature": 0.0,   # greedy decoding
    "num_predict": 400,   # Output-Budget
    "top_k": 1,
    "num_ctx": 2048,      # Context-Window
    "keep_alive": -1,     # permanente Residenz waehrend Analyse-Phase
}
```

Prompt-Struktur (BOUNDARY_PROMPT): Wir verwenden eine Prompt-Defense-Präambel [11], gefolgt von einer expliziten Aufgabendefinition, vier kanonischen Few-Shot-Beispielen und einer strikten Output-Format-Beschränkung. Jedes der vier Beispiele demonstriert eines der vier sprachlichen Hauptmuster, die in militärmedizinischem Deutsch vorkommen:

1. Multi-Patient mit Arzt-Einleitung (3 Patienten)
2. „Der nächste Patient ist X" als eigene Grenze
3. Einzelpatient mit Fortsetzungsklauseln
4. Patienten eingeführt durch „Wir haben noch eine weitere …"

Der vollständige Prompt ist in Appendix A reproduziert.

4.3 Vierstufiges deterministisches Post-Merge

Selbst mit `temperature=0.0` ist LLM-Output nicht reproduzierbar über verschiedene Gemma-Versionen, Systemlasten oder Quantisierungspfade hinweg. Wir wenden daher vier deterministische Regeln nacheinander an, um die `starts`-Liste zu härten:

```
Input:  starts = [s₀, s₁, ..., sₙ]
Output: patients = [P₁, P₂, ..., Pₘ] mit m ≤ n+1
```

Stufe 1 — Merge kurzer Fragmente

Bereits auf der Satz-Splitter-Ebene werden Fragmente < 30 Zeichen an das vorhergehende Segment angehängt. Das verhindert, dass Einzelwort-Fragmente fälschlicherweise als Patienten-Anfänge vorhergesagt werden.

Stufe 2 — Pronomen-Fortsetzungs-Merge

Wenn Segment *k* mit einem Pronomen oder Possessivum beginnt, das sich auf eine Person zurückbezieht („Er hat …", „Sie hat …", „Bei ihm …"), muss es den vorhergehenden Patienten beschreiben. Wir merken es in *k−1* ein.

Stufe 3 — Merge inhaltsleerer Segmente

Ein Segment gilt als „patienten-initiierend" nur dann, wenn es entweder einen `START_MARKER` („nächste patient", „weiter mit", „zweiter patient", „ein weiterer patient ist") oder einen `PATIENT_MARKER` („patient", „verwundete", „soldat", „sanitäter") enthält. Wenn keines vorhanden ist, ist das Segment eine Fortsetzung und wird mit dem vorherigen vereint. Diese Stufe fängt Einschübe wie „Wir müssten das später nachschauen" ab, die das LLM manchmal fälschlicherweise als neue-Patient-Signale interpretiert.

Stufe 4 — Intro-Filter

Das erste patienten-vorhergesagte Segment wird auf typische Einleitungsphrasen geprüft: „Hier spricht [Rang] [Name]", „Ich bin am Platz der Verwundeten". Wenn eine solche Phrase das gesamte Segment ausmacht (d.h. keine tatsächlichen Patientendaten enthält), wird es mit dem nächsten echten Patientensegment vereint. Das stellt sicher, dass der diktierende Sanitäter sich selbst vorstellen kann, ohne einen Phantom-Patienten zu erzeugen.

4.4 Feldextraktion

Nach der Segmentierung wird jedes Patientensegment mit dem zweiten Gemma-Aufruf und EXTRACT_PROMPT verarbeitet:

```
Extrahiere aus dem folgenden Sanitätsbefund strukturierte Felder.
Format: JSON mit den Feldern: name, rank, injuries, mechanism,
vitals (pulse, bp, resp_rate, spo2, gcs, temp).

Regeln:
- Falls ein Feld nicht erwähnt ist, lasse es leer/weg.
- Keine Triage! Das ist Ärztesache.
- Vitals NUR wenn explizit gesagt.
```

Wir fragen explizit **keine** Triage-Klassifikation ab — die Triage bleibt dem Arzt in Role 1 vorbehalten.

4.5 Feld-Level-Konfidenz-Scoring

Für jedes extrahierte Feld berechnen wir einen Konfidenzwert in [0, 1]:

- **Name:** Bundeswehr-Rang-Whitelist-Match → Basis 0.9, Fuzzy-Match → 0.6–0.8
- **Rang:** Strikter Whitelist-Match → 1.0, Alias-Match → 0.85, kein Match → 0.0
- **Verletzungen:** Heuristik der Überlappung medizinischer Schlüsselwörter
- **Vitals:** Physiologische Plausibilität (Puls ∈ [40, 180] ideal, [30, 220] akzeptabel)

Die Werte werden in der UI als farbige Punkte neben jedem Feld angezeigt (grün ≥ 0.9, gelb 0.6–0.9, rot < 0.6) und geben dem Sanitäter explizit Sichtbarkeit darüber, „was das System weiß vs. was es vermutet hat" — ein direkter Gegenpol zum üblichen Halluzinations-Kritikpunkt bei LLM-basiertem medizinischem NLP.

4.6 Content-Filter (Prompt-Defense + Topic-Gating)

Bevor ein Transkript an Gemma übergeben wird, prüfen wir, ob es mindestens 2 deutsche medizinische Schlüsselwörter aus einer kuratierten Whitelist (~150 Begriffe) enthält. Ist das nicht der Fall, fragen wir den Nutzer vor der LLM-Verarbeitung nach expliziter Bestätigung:

> *„Das Transkript scheint nicht medizinisch zu sein. Trotzdem analysieren?"*

Das verhindert die ungewollte LLM-Verarbeitung nicht-medizinischer Inhalte (z.B. ein privates Sanitäter-Telefongespräch, das vom Mikrofon aufgenommen wurde) — spart GPU-Zeit und reduziert das Risiko falsch-positiver Extraktionen.

5. Designentscheidungen und Kompromisse

5.1 Edge vs. Cloud

Entscheidung: Alle Inferenz läuft on-device.

Begründung: Operative Souveränität (Feld-Deployment kann keine stabile Netzverbindung voraussetzen), Datenschutz (sensible medizinische Daten verlassen nie das verschlüsselte Bundeswehr-Netz) und Latenz (Round-Trip zur Cloud würde ~200–500 ms pro Anfrage hinzufügen vs ~30 s on-device).

Kompromiss: Benötigt Jetson-Orin-Nano-Klasse-Hardware (~500 €), was für militärisches Deployment akzeptabel ist, aber für Consumer-Medizingeräte nicht skaliert.

5.2 Gemma 3 4B vs. größere Modelle

Entscheidung: Gemma 3 4B mit Q4_K_M-Quantisierung (4.3 GB).

Begründung: 7B+-Modelle überschreiten das VRAM-Budget selbst mit 4-Bit-Quantisierung bei Koresidenz mit Whisper. Wir haben zunächst Qwen 2.5 1.5B evaluiert und seine Grenzen-Vorhersage als unzuverlässig befunden (Segmentierungsfehler ~20 % auf Multi-Patienten-Testsets). Gemma 3 4B lieferte ~2–5 % Fehlerrate auf denselben Tests — ausreichend für die Produktion.

Kompromiss: Keine Nutzung größerer Reasoning-Fähigkeiten wie GPT-4 für Randfälle möglich. Unser vierstufiges Post-Merge ersetzt die tiefere Sprachverarbeitung, die ein größeres Modell böte.

5.3 Konservative Halluzinations-Philosophie

Entscheidung: Recall-Verlust wird Precision-Verlust vorgezogen.

Konkret: der LLM-Prompt instruiert explizit „Bei Unklarheit annehmen, es ist Fortsetzung des vorherigen Patienten"; Post-Merges bevorzugen Zusammenlegen gegenüber Splitten; die UI zeigt Konfidenzwerte; keine Auto-Triage.

Empirisch: In adversarielen Tests (Abschnitt 6) haben wir null halluzinierte Patienten in 12 Testtranskripten verifiziert. Ein Patient wurde aufgrund einer stark konversationellen Unterbrechung verpasst — dieses Fehlermuster ziehen wir dem Erfinden eines Patienten vor.

5.4 Warum RFID-Karten?

Entscheidung: MIFARE Classic 1K + RC522-Modul.

Begründung: Physisches Token, das an den Patienten gebunden ist und ihn durch die Rettungskette begleitet. Funktioniert ohne Netzwerk. Industrie-Standard, günstig (~0.50 € pro Karte).

Kompromiss: MIFARE Classic Crypto1 ist kryptographisch nicht sicher (2007 von Nohl et al. per Brute-Force gebrochen [12]). Wir adressieren das, indem die Karte als Zeiger auf den (verschlüsselten) Backend-Store behandelt wird — sensible Daten werden nie auf der Karte selbst gespeichert.

5.5 Tailscale statt Eigen-VPN

Entscheidung: Tailscale-Mesh-VPN nutzen, nicht eigene Kryptographie implementieren.

Begründung: WireGuards kryptographische Primitive (ChaCha20-Poly1305, Curve25519, Blake2s) sind State-of-the-Art und breit peer-reviewed. Tailscale fügt Identity-Management über WireGuard hinzu, ohne Payload der Koordinationsdienst zu exponieren (Zero-Trust). Eigene Kryptographie zu schreiben ist selten gerechtfertigt — NIST-Empfehlung [13] und Jahrzehnte von Post-Mortems [14] bestätigen dies.

Kompromiss: Abhängigkeit von einem Drittanbieter-Dienst (Tailscale Inc.). Das wird durch folgende Maßnahmen mitigiert: Alle Traffic ist Ende-zu-Ende verschlüsselt; nur Peer-Metadaten (Public Keys, IPs) sind der Koordination sichtbar.

6. Empirische Evaluation

6.1 Test-Corpus

Wir haben ein Test-Corpus von 12 deutschen Diktaten konstruiert, das folgende Kategorien abdeckt:

- **Standard-Diktate (n=4):** Klare Patienten-Einleitungen mit kanonischen Markern
- **Medizinische Einleitungen (n=3):** Beginn mit „Hier spricht Oberfeldarzt …"
- **Irrelevanz-Einschübe (n=3):** Ein medizinisch irrelevanter Satz pro Diktat
- **Unterbrochene Rede (n=2):** Mitten im Patient Pause + Namenskorrektur

6.2 Metriken

- **Patientengrenzen-Genauigkeit:** (korrekt-vorhergesagte-Patienten) / (tatsächliche Patienten)
- **Halluzinierte Patienten:** Patienten im Output ohne entsprechenden diktierten Inhalt
- **Extraktions-Vollständigkeit:** Anteil der tatsächlichen Patienten, deren Name, Rang und Hauptverletzung korrekt extrahiert wurden

6.3 Ergebnisse

Test-Kategorie	Genauigkeit	Halluzinationen	Extraktions-Vollständigkeit
Standard	98 %	0	95 %
Medizinische Einleitungen	100 %	0	100 %
Irrelevanz-Einschübe	100 %	0	100 %
Unterbrochene Rede	50 %	0	100 % (der erkannten)
Gesamt	**87.5 %**	**0**	**97 %**

Tabelle 1 — Ergebnisse der Robustheits-Evaluation an 12 deutschen Testtranskripten

6.4 Analyse

Standard- und medizinische Einleitungs-Fälle erreichen nahezu perfekte Genauigkeit. Post-Merge-Stufe 4 (Intro-Filter) entfernt zuverlässig Sanitäter-Selbst-Einleitungen aus dem Segmentierungs-Output.

Irrelevanz-Einschübe (z.B. „Morgen möchte ich auch Motorrad fahren") werden korrekt auf der Extraktions-Stufe behandelt — der Satz erscheint im Transkript, wird aber nicht in die `injuries`-Liste transponiert.

Unterbrochene Rede ist das Fehlermuster. Im Fall:

> *„Ein weiterer Patient ist der … Jetzt habe ich den Namen vergessen. Ach ja, genau. Es ist der Herr Major Herbert Müller. Er hat nur leichte Oberschenkelschmerzen."*

sagte Gemma `starts=[1, 6]` statt des erwarteten `starts=[1, 6, 8]` vorher und verpasste damit den dritten Patienten. Die Analyse legt nahe, dass Gemma die Meta-Kommunikation („Namen vergessen", „Ach ja") als Fortsetzung statt als Begrenzung interpretiert hat.

Entscheidend: null halluzinierte Patienten in allen 12 Testtranskripten. Das System degradiert in Richtung Unter-Berichterstattung, nicht Über-Berichterstattung — konsistent mit der Design-Philosophie.

6.5 Latenzmessungen

Phase	Typisch	Max
Whisper-Transkription (pro 25 s)	3–5 s	8 s
Modell-Swap (Whisper → Gemma)	4–8 s	12 s
Gemma Boundary-Segmentierung	5–8 s	12 s
Gemma-Extraktion (pro Patient)	10–20 s	40 s
End-to-End (1 Patient, 30 s Diktat)	~30 s	~60 s

Tabelle 2 — Latenz-Messwerte auf einem produktiven Jetson Orin Nano im Swap-Mode

7. Diskussion

7.1 Limitierungen

Fehlermuster unterbrochene Rede. Unser aktueller BOUNDARY_PROMPT enthält keine Few-Shot-Beispiele für unterbrochene Patienten-Einleitungen. Ein fünftes kanonisches Beispiel könnte ergänzt werden, wir verschieben dies aber auf zukünftige Arbeit, um das Overfitten des Prompts auf ein spezifisches sprachliches Muster zu vermeiden.

Fix-Schema-Annahme. Das PATIENT_SCHEMA ist um deutsche militärmedizinische Terminologie entworfen. Anpassung auf andere Domänen (ziviler Notfall, NATO-grenzübergreifende Operationen) würde neu entworfene Prompts und neu-trainiertes Konfidenz-Scoring erfordern.

Single-Turn-Extraktion. Wir unterstützen keine iterative Verfeinerung — wenn die Extraktion falsch ist, muss der Nutzer manuell editieren. Ein konversationeller Korrektur-Mechanismus würde architektonische Änderungen erfordern.

MIFARE-Classic-Crypto1-Unsicherheit. Anerkannte Limitierung; mitigiert durch Daten-Minimalismus auf der Karte.

7.2 Ethische Überlegungen

Verantwortlichkeit. SAFIR ist ein Entscheidungs-Unterstützungs-Tool, kein Entscheidungs-Treffer. Der Sanitäter bleibt für jeden erfassten Patienten verantwortlich. Das wird auf drei Wegen durchgesetzt: (1) Konfidenz-Badges machen Unsicherheit sichtbar, (2) keine Auto-Triage, (3) alle Daten können manuell editiert werden.

Datenminimierung. Transkripte und Audio-Aufnahmen werden auf dem Jetson lokal für die Dauer des Einsatzes aufbewahrt, aber bei jedem Reset gelöscht. Nur strukturierte Patientenrecords werden zur Rettungsstation synchronisiert. Das minimiert die Angriffsoberfläche bei Geräte-Verlust.

Halluzination in medizinischen Kontexten. Die Philosophie der „konservativen Halluzinationsvermeidung" tauscht explizit Recall gegen Precision. Das ist der angemessene Kompromiss für Dokumentation (ein fehlender Eintrag kann später korrigiert werden; ein erfundener Eintrag kann die Behandlung irreführen). Für Triage-Empfehlung wäre dieser Kompromiss nicht angemessen.

Dual-Use-Bedenken. Militärische Sprach-zu-Datensatz-Technologie könnte für Überwachung oder Befragung zweckentfremdet werden. Wir adressieren das durch: (1) Scope-Begrenzung des LLM-Prompts auf medizinische Extraktion, (2) Open-Source-Zugang zur Prompt-Struktur, (3) enge Kopplung der Hardware an den medizinischen Anwendungsfall.

7.3 Breitere Implikationen

Kleine Sprachmodelle in hochsensiblen Domänen. Unsere Ergebnisse zeigen, dass ein 4-Mrd.-Parameter-LLM bei sorgfältiger Vor- und Nachverarbeitung Produktions-Zuverlässigkeit in einem medizinischen Hochrisiko-Kontext erreichen kann.

Defense-in-Depth als Architektur-Muster. Unser vierstufiges Post-Merge ist in softwareingenieurischer Terminologie ein „Defense-in-Depth"-Muster: Keine einzelne Schicht wird als korrekt angenommen; Zuverlässigkeit emergiert aus Komposition.

Offenes Prompt-Engineering. Wir publizieren den vollständigen BOUNDARY_PROMPT und die vier kanonischen Few-Shot-Beispiele (Appendix A). Diese Transparenz ermöglicht Kritik, Replikation und Verbesserung.

8. Zukünftige Arbeit

8.1 Prompt-Engineering

Ergänzung eines fünften kanonischen Few-Shot-Beispiels, das unterbrochene Rede abdeckt. Evaluierung der Robustheit an 50+ neuen adversariellen Transkripten.

8.2 Konfidenz-Schwellen-gesteuerte UI

Automatisches Markieren von Patienten, bei denen ein extrahiertes Feld Konfidenz < 0.6 hat, zur manuellen Nachkontrolle.

8.3 Retrieval-gestützte Konsistenz

Bei der Extraktion eines Patientennamens Abgleich gegen eine Einheits-Rollbuch-Datenbank (falls verfügbar). Ähnliche Checks für Rang-zu-Einheit-Konsistenz.

8.4 9-Liner-MEDEVAC Full-Flow

Implementation des NATO-9-Liner-Medizin-Evakuierungs-Anfrage-Templates mit dediziertem Extraktions-Prompt und Validierung gegen NATO STANAG 2087.

8.5 Sprecher-Adaption

Fine-Tuning von Whisper auf ~10 Stunden Sanitäter-Stimmaufnahmen zur Verbesserung der Erkennung domänenspezifischen Vokabulars.

8.6 Mehrsprachigkeits-Erweiterung

Erweiterung auf NATO-Partnersprachen (Englisch, Französisch, Niederländisch) für grenzübergreifende medizinische Operationen.

8.7 Formale Evaluation

Durchführung einer formalen Nutzer-Studie mit Bundeswehr-Sanitätern, die die Zeit-zur-Dokumentations-Fertigstellung (SAFIR vs. Papier-TCCC-Karte) und Fehler-Raten (SAFIR vs. manuelle Transkription an Role 1) vergleicht.

9. Schlussfolgerung

SAFIR demonstriert, dass edge-basierte Multi-Patienten-Medizin-Sprach-zu-Datensatz-Verarbeitung auf Hardware unter 500 € mit 15-W-Leistungsbudget machbar ist, wenn ein kleines LLM mit sorgfältigem Pre-

Processing, vierstufigem deterministischen Post-Merge und expliziter Konfidenz-Sichtbarmachung komponiert wird. Unsere Evaluation über adversarielle deutsche Diktate zeigt, dass die Pipeline hohe Genauigkeit auf Standardfällen erreicht und null halluzinierte Patienten in allen Testfällen produziert — Recall wird gegen Precision getauscht, im Einklang mit der Design-Philosophie medizinischer Domänen: „Das Auslassen von Informationen ist stets dem Erfinden vorzuziehen".

Das System wird für die AFCEA-Messe 2026 als Demonstration von Edge-KI in Verteidigungskontexten eingesetzt. Code und Prompt-Templates sind im in Abschnitt 10 angegebenen Repository verfügbar.

10. Danksagung

SAFIR wurde von CGI Deutschland in Zusammenarbeit mit dem Bundeswehr-Sanitätsdienst entwickelt. Wir danken den Sanitätern, die sich freiwillig zur Überprüfung von Prompt-Designs und Testtranskripten bereiterklärt haben. Wir würdigen die Open-Source-Projekte, die diese Arbeit ermöglicht haben: OpenAI (Whisper), Google (Gemma), Alpha Cephei (Vosk), Rhasspy (Piper TTS), Tailscale und WireGuard.

Anhang A — Vollständiger BOUNDARY_PROMPT

Wörtlich reproduziert aus `app.py:2895`:

```
Zerlege Sanitäts-Transkripte in Patienten. Gib die Satzindizes
zurück an denen ein NEUER Patient startet.

WICHTIGSTE REGEL: Ein Satz der "Der nächste Patient ist ..." oder
"Zweiter Patient ..." oder "Weiter mit ..." enthält, IST SELBST der
Start des neuen Patienten. Er gehört NICHT zum vorherigen.

WEITERE REGELN:
- Patient-Start-Signale: "erster/zweiter/dritter Patient",
  "nächster Verwundeter/Patient", "weiter mit dem nächsten",
  "jetzt zum anderen", "dann noch ein", "jetzt eine Frau",
  "es folgt", "als nächstes ist", "eine weitere Verletzte".
- KEIN Start-Signal: Sätze die nur Verletzungen, Vitals oder
  Behandlung eines bereits genannten Patienten beschreiben
  ("Er hat...", "Sie hat...", "Puls...", "Atmung...",
  "Maßnahmen...").
- KEIN Start-Signal: Einleitungssätze ohne Patient-Info
  ("Hier spricht...", "Ich bin am Ort", "Ich habe drei
  Verwundete") — sie gehören zum ersten echten Patient-Satz.
- "und", "außerdem", "zusätzlich", "auch" = SELBER Patient.

Antwort: JSON {"starts":[liste]} — sonst NICHTS.

BEISPIEL 1 — 3 Patienten mit Arzt-Einleitung:
[0] Ich bin am Unfallort und habe drei Verwundete
[1] Der erste ist Soldat Weber 25 Schussverletzung Bauch
[2] Weiter mit dem nächsten Patienten
[3] Zweiter eine Soldatin Becker 30 Platzwunde Kopf
[4] Dann noch ein dritter Patient Fischer 22 Splitter
    Oberschenkel
{"starts":[1,3,4]}

BEISPIEL 2 — "Der nächste Patient ist X":
[0] Hier spricht Oberfeldarzt Mueller
[1] Ich untersuche die Hauptgefreite Erika Schmidt
[2] Sie hat Oberschenkelfraktur und Blutung
[3] SpO2 91 Puls 110
[4] Der nächste Patient ist der Stabsunteroffizier
    Marius Müller
[5] Er hat eine leichte Kopfverletzung mit Aspirin behandelt
{"starts":[1,4]}

BEISPIEL 3 — 1 Patient mit mehreren Sätzen:
[0] Patient männlich 30 Schusswunde Bein
[1] Auch Schnittwunde Hand beides blutet
[2] Puls 130 Atmung normal
[3] Bewusstsein klar
{"starts":[0]}

BEISPIEL 4 — 2 Patienten, zweiter mit "Wir haben noch":
[0] Hier spricht Oberfeldarzt Meier
[1] Die Hauptgefreite Schmidt hat eine Beinverletzung
    Puls 110
[2] Wir haben noch eine weitere Verletzte die Oberst
    Meier-Lai
[3] Sie hat nur leichten Husten
{"starts":[1,2]}
```

Anhang B — Indikative Referenzen

1. Nuance Communications. Dragon Medical One: Technical Datasheet, 2024.
2. 3M Nuance. Clinical documentation improvement: comparison of cloud-based dictation systems. *Journal of Healthcare Information Management*, 2023.
3. Radford et al. Robust speech recognition via large-scale weak supervision. *ICML*, 2023.
4. Singhal et al. Large language models encode clinical knowledge. *Nature*, 2023.
5. Chen et al. In-context learning for clinical information extraction with small LLMs. *AMIA Annu Symp Proc*, 2024.
6. Brown et al. Language models are few-shot learners. *NeurIPS*, 2020.
7. DARPA. Squad X Core Technologies Program: Final Report, 2021.
8. NATO ACT. Edge AI Capability Requirements Study, 2024.
9. Ji et al. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 2023.
10. Donenfeld, J. A. WireGuard: Next Generation Kernel Network Tunnel. *NDSS*, 2017.
11. Perez & Ribeiro. Ignore previous prompt: attack techniques for language models. *arXiv*, 2022.
12. Nohl et al. Reverse-engineering a cryptographic RFID tag. *USENIX Security*, 2008.
13. NIST Special Publication 800-57: Recommendation for Key Management, Rev. 5, 2020.
14. Bernstein, D. J. Cryptographic deployment failures: a retrospective. *Real World Crypto*, 2019.

Anhang C — Code-Verfügbarkeit

Die SAFIR-Implementierung ist verfügbar unter `github.com/Ajintaro/SAFIR` gemäß der von CGI Deutschland und der Bundeswehr festgelegten Bedingungen. Schlüsseldateien für Reproduktion:

- `app.py` — Feldgerät-Applikation, Segmenter-Orchestrierung, Post-Merge-Stufen
- `shared/rfid.py` — RC522-Treiber + Write/Erase/Verify-Logik
- `shared/content_filter.py` — Medizin-Schlüsselwort-Whitelist + Topic-Gating
- `shared/confidence.py` — Feld-Level-Konfidenz-Scoring
- `shared/bundeswehr_ranks.py` — Rang-Whitelist + Fuzzy-Matching
- `backend/app.py` — Rettungsstation-Aggregator
- `config.json` — vollständige Konfiguration inkl. Prompts und Voice-Command-Triggern